

Mandrill 2.0 编译器完整实践实验报告

2026 年 5 月 23 日

摘要

本实验围绕 Mandrill 2.0 语言编译器的完整实现展开，重点不是堆叠功能点，而是把词法切分、递归下降语法分析、基于符号表的语义检查以及面向虚拟机的代码生成这四个算法模块串成一条稳定的流水线。仓库中提供的样例程序、异常测试、批处理脚本和单元测试共同构成了验证闭环：正常样例用于确认算法输出是否与标准答案一致，异常样例用于确认错误是否在合适的阶段被拦截，模拟器测试则用于确认生成的汇编代码是否能够按预期执行。实验结果表明，系统能够正确识别关键字、标识符、常量和运算符，能够按照文法层次重建程序结构，能够在编译期发现作用域、类型和上下文错误，并能够把合法程序翻译成可运行的虚拟机指令。

1 实验目的与对象

这次实验对应仓库中的 EX1 到 EX4 四个阶段，目标不是把编译器一次性写完，而是按层实现并验证。前端部分解决“程序长什么样”，中间部分解决“这些名字和类型在当前上下文里是否成立”，后端部分解决“如何把正确的程序翻成可以跑的指令”。这条路径之所以适合课程实验，是因为每一层都有明确输入和输出，出错时也能迅速定位。

Mandrill 2.0 本身是一门面向课程实验设计的命令式语言，支持整数、数组、字符串、字符常量、函数、条件语句、循环语句、跳转语句以及输入输出操作。仓库中的参考手册和语法文件给出了这门语言的完整词法和文法约束，EX1 到 EX4 则分别实现了词法分析器、语法分析器、语义检查器以及编译器与模拟器。实验对象就是这些真实实现及其样例数据，不是抽象教材例子。

2 算法设计

Mandrill 编译器的核心算法可以概括为四步：先把字符流切成 token，再把 token 组织成语法结构，随后把结构放进符号表里做上下文检查，最后把通过检查的结构变成目标指令。为了让实现逻辑更清楚，下面把四个核心算法写成伪代码。

词法分析算法

Listing 1: 词法分析算法伪代码

```

1 input: character stream src
2 output: token list tokens
3
4 while src not end:
5     read one character c
6     if c is whitespace:
7         skip
8     else if c is digit:
9         read consecutive digits
10        emit integer token
11    else if c is letter or '_':
12        read letters, digits and '_'
13        if lexeme in keyword table:
14            emit keyword token
15        else:
16            emit identifier token
17    else if c is single quote:
18        read character constant
19        emit char token
20    else if c is double quote:
21        read string constant
22        emit string token
23    else if c starts operator or delimiter:
24        resolve one-char or two-char token
25    else:
26        record illegal character
27 emit EOF token
28 return tokens

```

这个算法的关键不在于分支多，而在于扫描顺序必须稳定。尤其是关键字和标识符的区分，必须先完整读出词素，再查关键字表，否则像 ‘write1’ 这样的名字会被误判成保留字前缀。EX1 中的实现正是沿着这个思路写的：逐字符读取、维护行列信息、用前瞻处理 ‘==’、‘!=’、‘<=’、‘>=’，最后统一输出 token 序列。

递归下降语法分析算法

Listing 2: 递归下降语法分析伪代码

```

1 parseProgram():

```

```

2   while current token != EOF:
3       if current token is FUNC:
4           parseFunctionDef()
5       else:
6           parseStatement()
7
8   parseExpression():
9       parseEquality()
10
11  parseEquality():
12      parseRelational()
13      while next token is == or !=:
14          consume operator
15          parseRelational()
16
17  parseRelational():
18      parseAdditive()
19      while next token is <, <=, > or >=:
20          consume operator
21          parseAdditive()
22
23  parseAdditive():
24      parseMultiplicative()
25      while next token is + or -:
26          consume operator
27          parseMultiplicative()
28
29  parseMultiplicative():
30      parsePrimary()
31      while next token is *, / or %:
32          consume operator
33          parsePrimary()

```

语法分析这里最容易写乱的地方是表达式优先级。EX2 的实现没有采用回溯，而是把表达式拆成几层，每层只处理一种优先级，这样就能保持递归下降的简单性，同时保证 ‘* /

语义检查算法

Listing 3: 语义检查算法伪代码

```

1   visitProgram(node):

```

```

2      build global symbol table
3      visit function definitions and global statements
4
5  visitFunctionDef(node):
6      enter function scope
7      register parameters
8      visit function body
9      exit function scope
10
11 visitDeclaration(node):
12     if name already exists in current scope:
13         report duplicate definition
14     else:
15         register symbol type and scope
16
17 visitJump(node):
18     if break/continue outside loop:
19         report error
20     if return outside function:
21         report error
22     if return type mismatches function declaration:
23         report error
24
25 visitCall(node):
26     check function exists
27     check argument count
28     check argument types against parameter types

```

语义检查的本质是维护状态。当前是不是在函数里、当前是不是在循环里、某个名字属于全局还是局部、变量到底是整数还是数组，这些状态决定了一条语句是否成立。EX3 和 EX4 都使用了符号表和作用域栈来维护这些信息，前者负责在语法树遍历时做严格约束，后者则把收集到的符号信息带入代码生成阶段。这样做的好处是很直接的：很多本来会到运行时才出错的问题，在编译期就能被挡下来。

代码生成算法

Listing 4: 代码生成算法伪代码

```

1  compileProgram(program):
2      jump to main entry
3      compile all function definitions
4      compile top-level statements

```

```
5      emit halt jump
6
7  compileExpression(expr):
8      if expr is constant:
9          emit immediate load
10     else if expr is variable:
11         emit load from global or local frame
12     else if expr is function call:
13         push arguments in reverse order
14         emit call instruction
15     else if expr is binary operator:
16         compile left operand
17         compile right operand
18         emit eval instruction
19
20 compileStatement(stmt):
21     if stmt is assignment:
22         compute right value
23         write back to target address
24     if stmt is if or while:
25         create labels and branch instructions
26     if stmt is return:
27         compile return expression
28         emit ret
```

代码生成阶段最需要注意的是调用约定和控制流。函数入口必须先处理栈帧，函数调用前参数要按约定入栈，数组访问要先计算地址再读写，条件和循环要先生成标签再回填跳转。EX4 的 ‘CompilerImpl’ 和模拟器测试说明，这一层不是简单“把树吐成文本”，而是要对栈、堆、标签和返回地址都保持一致的理解。

3 实现过程与结果

EX1 到 EX4 的实现顺序很自然：先切 token，再排结构，再做语义检查，最后发射指令。真正难的地方不是把代码写满，而是让前一层输出刚好喂给后一层。EX1 中的词法器是整个链条的入口，程序从 UTF-8 输入流逐字符读取，维护行号和列号，识别关键字、标识符、字面量以及单字符和双字符运算符。EX2 把 token 还原成程序结构，能判定函数定义、语句块、条件语句、循环语句和表达式是否合法。EX3 进入语义阶段后，重点变成了上下文一致性：变量是否声明、数组是否被当作整数使用、函数调用参数个数是否正确、‘break’ 和 ‘continue’ 是否出现在循环中、‘return’ 是否出现在函数里。EX4 则把这些通过检查的结构翻成汇编，并通过模拟器确认它真的能跑。

仓库中的批量测试脚本把这一过程固定下来。EX1 用 ‘mandrill-src’ 下的 6 个样例和标准 lexerout 比对；EX2 用 5 个正常样例和 5 个异常样例分别验证 Pass 和 Error；EX3 则在 6 个正常样例之外再检查 15 个语义错误样例；EX4 用 5 组输入/答案文件做端到端验证。相比“看起来像对了”的手工检查，这种脚本式验证更像真正的实验记录，因为输入、输出和判定规则都明确写在仓库里。

为了保留实现证据，下面摘录三段关键代码和脚本逻辑。它们不是为了堆材料，而是为了说明算法确实落在了真实代码里：词法阶段依赖关键字表和扫描循环，语义阶段依赖循环和函数上下文，后端则依赖编译入口和批量执行脚本。

关键代码与脚本摘录

Listing 5: EX1 词法分析器中的关键字表与扫描循环

```
1 static {
2     keywords = new HashMap<>();
3     keywords.put("if", TokenType.IF);
4     keywords.put("else", TokenType.ELSE);
5     keywords.put("while", TokenType.WHILE);
6     keywords.put("read", TokenType.READ);
7     keywords.put("put", TokenType.PUT);
8     keywords.put("write", TokenType.WRITE);
9     keywords.put("get", TokenType.GET);
10    keywords.put("func", TokenType.FUNC);
11    keywords.put("global", TokenType.GLOBAL);
12    keywords.put("local", TokenType.LOCAL);
13    keywords.put("return", TokenType.RETURN);
14    keywords.put("break", TokenType.BREAK);
15    keywords.put("continue", TokenType.CONTINUE);
16 }
17
18 public List<Token> scanTokens() {
19     tokens = new ArrayList<>();
20     line = 1;
21     column = 0;
22     while (true) {
23         currentChar = reader.read();
24         if (currentChar == -1) {
25             break;
26         }
27         column++;
28         scanToken();
```

```

29     }
30     tokens.add(new Token(TokenType.EOF, "<EOF>", line, column + 1));
31     return tokens;
32 }

```

Listing 6: EX3 语义检查器中的循环和返回约束

```

1  @Override
2  public MandrillType visitJumpStmt(MandrillParser.JumpStmtContext ctx) {
3      if (ctx.Break() != null) {
4          if (loopNestingLevel == 0) {
5              error(ctx, "break statement must be inside a loop");
6          }
7      } else if (ctx.Continue() != null) {
8          if (loopNestingLevel == 0) {
9              error(ctx, "continue statement must be inside a loop");
10         }
11     } else if (ctx.Return() != null) {
12         if (functionNestingLevel == 0) {
13             error(ctx, "return statement must be inside a function");
14         }
15     }
16     return MandrillType.UNKNOWN;
17 }

```

Listing 7: EX4 编译器入口和端到端测试

```

1  public String compile(InputStream inputStream) throws IOException {
2      CompileContext context = Compiler.frontend(inputStream);
3      AssemblyBuilder builder = new AssemblyBuilder(context.table());
4      builder.compileProgram(context.tree());
5      return builder.build();
6  }
7
8  @Test
9  void testAddProgram() throws Exception {
10     String asm = ""
11         geti 0
12         geti 0
13         eval 65537
14         puti 0
15         jump 4294967295
16         "";

```

```
17     assertEquals("8", runAssembly(asm, "3 5"));
18 }
```

Listing 8: EX2 和 EX4 的批量验证脚本摘录

```
1 for filec in $(ls $normaldir/*.mds); do
2     fileout="PASS.txt"
3     echo "Pass" >$fileout
4     cp $filec data.mds
5     timeout 2 $CCHK <data.mds 1>data.parserout
6     diff data.parserout "$fileout" >parserout.diff.txt
7 done
8
9 for filec in $(ls $normaldir/*.mds); do
10     cp $filec data.mds
11     cp $filein mandrill.in
12     timeout 2 $MDC data.mds data.asm
13     timeout 2 $MDS data.asm mandrill.in data.out
14     diff data.out "$fileans" >ans.diff.txt
15 done
```

4 数据记录与分析

仓库中的样例规模是明确的：EX1 有 6 个词法样例，EX2 有 5 个正常样例和 5 个异常样例，EX3 有 6 个正常样例和 15 个语义错误样例，EX4 有 5 组输入和答案文件。这个规模不算夸张，但足够覆盖 Mandrill 2.0 的主干语法和常见错误类型。更重要的是，样例分布和实验阶段是一一对应的，所以测试结果能直接反映某一层算法是否稳定，而不是混杂地反映整条流水线。

从记录上看，EX1 的关键在 token 边界，EX2 的关键在优先级和语句归约，EX3 的关键在作用域和类型状态，EX4 的关键在调用约定和跳转标签。这样的划分让调试思路很清楚：如果 lexer 出错，后面的阶段都不用看；如果 parser 能过但语义失败，问题多半在符号表或上下文；如果编译结果和答案不一致，就要回到地址计算和控制流上排查。批量脚本的价值就在这里，它把每一层的“对不对”变成可复核的文本输出，而不是靠主观判断。

从工程角度看，这份实现比较稳的地方在于“前面收紧、后面再放开”。EX3 的符号收集把函数、参数和局部变量的布局先固定下来，EX4 再根据这个布局生成汇编，因此函数调用、递归和数组访问都能保持一致的帧结构。单元测试则把指令级行为单独拉出来，比如加法、条件跳转和函数返回，这样当端到端样例不通过时，可以先判断是编译器错了，还是模拟器本身的指令语义错了。

5 结论

Mandrill 2.0 编译器实验最终完成了一条完整且可验证的实现链路。这个链路里最有价值的不是某个类写得多复杂，而是每一层算法都和下一层严格对齐：词法把字符变成 token，语法把 token 变成结构，语义把结构放回上下文里检查，代码生成再把正确结构翻译成可执行指令。这样的实现方式朴素，但对课程实验来说足够可靠，也最容易通过样例和脚本证明正确性。

从结果上看，仓库中的正常样例能够稳定通过，异常样例能在对应阶段被拦住，端到端执行也能得到与标准答案一致的输出。这说明该实现已经把 Mandrill 2.0 的主要语法、语义和执行机制串起来了，达到了实验要求。

参考文献

- [1] docs/Mandrill 2.0 项目及语法概览.md, Mandrill 2.0 语言参考手册。
- [2] Terence Parr. ANTLR 4 Documentation and Runtime. <https://www.antlr.org/>
- [3] Compilers 仓库中的各阶段实现文件，包括 EX1、EX2、EX3、EX4 的源代码、脚本与测试样例。